Desafio Extra de Projeto
e
Análise de Algoritmos

Aluno: Arthur Souza Ribeiro
Curso: Sistema de Informação
Periodo: 4º Periodo

Professor: Alexandre Barbosa de Souza

2026
Vitoria – ES

1- Introdução

A DataFlow Corp. opera na gestão e no processamento contínuo de grandes volumes de dados. À medida que o volume de dados do sistema cresce, gargalos operacionais na camada de processamento afetam diretamente o throughput global da arquitetura.

Para assegurar a eficiência operacional e o cumprimento dos acordos de nível de serviço (SLA), foram mapeadas duas operações críticas na infraestrutura da plataforma:

1. Validação de Duplicatas: Processo de varredura e checagem de dados para identificação e eliminação de registros redundantes antes da persitência final.

2. Ordenação por Distância/Valor: Algoritmo de ordenação focado na estruturação sequencial de conjuntos de dados ordenados por métricas de proximidade geográfica ou valores financeiros para priorização de rotas e processamento financeiro.

2- Fundamentação Teórica

A análise de complexidade de algoritmos utiliza a notação assintótica para descrever o comportamento do tempo de execução e do uso de memória à medida que a entrada n cresce:

- Notação *O* (Big-O / Limite Superior): Define o pior cenário possível para o tempo de execução de um algoritmo. *Intuição:* "No pior caso, o algoritmo não gastará mais tempo do que este limite.

- Notação Ω (Big-Omega / Limite Inferior): Define o melhor cenário possível de desempenho. *Intuição:* "No melhor caso, o algoritmo gastará no mínimo este tempo.

- Notação Θ (Big-Theta / Limite Justo): Ocorre quando os limites superior e inferior coincidem, caracterizando o comportamento exato em todos os cenários. *Intuição:* "O algoritmo sempre se comporta exatamente de acordo com esta ordem de grandeza.

Algoritmo	Melhor caso	Pior caso	Caso médio	Espaço extra
Busca Linear	Ω(1)	O(n)	Θ(n)	O(1)
Busca Binária (dados ordenados)	Ω(1)	O(log n)	Θ(log n)	O(1)
Selection Sort	$\Omega(n^2)$	$O(n^2)$	$\Theta(n^2)$	O(1)
Merge Sort	Ω(n log n)	O(n log n)	Θ(n log n)	O(n)

3- Metodologia Experimental

Processador: AMD Ryzen 5 5500 (3.60 GHz)

RAM instalada: 16,0 GB (utilizável: 15,9 GB)

Placa gráfica: NVIDIA GeForce RTX 2060 (6 GB)

Java: 27

IDE/Terminal: Vscode

- Tamanhos: n = 100; 1.000; 5.000; 10.000; 50.000; 100.000 (e 500.000, se rodou). Selection Sort não foi executado em 500.000 por ser inviável em tempo.
- Tipos de dados: aleatório (semente fixa, valores em [0, 10n)) e ordenado (valores pares crescentes).
- Procedimento: 4 execuções por caso, descarte da 1ª (warm-up da JVM), média das 3 restantes; tempo com *System.nanoTime()*.
- Métricas: tempo, comparações e trocas/movimentos. No Selection Sort conta-se 1 troca por swap; no Merge Sort contam-se as cópias para o vetor auxiliar e de volta.

- Buscas: 1000 consultas por execução (metade de chaves presentes, metade ausentes = pior caso); reporta-se a média por consulta. Para a Busca Binária, o vetor foi ordenado antes, fora da medição.
- Validação: vetor ordenado e mesmo conteúdo do original (ordenações); índice correto ou −1 (buscas).

4- Resultados

Tabela 1: Tempo Médio de Execução em Nanorsegundos (ns)

A	B	C	D	E	F	G	H
Algoritmo	Tipo de Dado	n=100	n=1.000	n=5.000	n=10.000	n=50.000	n=100.000
Busca Linear	Aleatório	273,7	207	789,3	1.537,30	10.885,70	22.028,00
	Ordenado	100,7	162,3	1.096,00	1.616,30	10.748,30	23.235,30
Busca Binária	Aleatório	79,3	78,3	46,7	111	51,3	67,7
	Ordenado	59	39,3	47	86	62	67
Selection Sort	Aleatório	187.666,70	2.155.766,70	5.461.633,30	21.467.566,70	690.616.500,00	2.761.925.600,0
	Ordenado	119.700,00	186.900,00	5.033.666,70	18.284.800,00	698.470.500,00	2.779.988.833,3
Merge Sort	Aleatório	55.966,70	95.033,30	382.266,70	1.477.966,70	4.837.433,30	10.906.400,00
	Ordenado	5.333,30	67.066,70	239.166,70	317.000,00	1.828.700,00	4.049.100,00

Tabela 2: Média de Comparações e Trocas/Movimentos

A	B	C	D	E	F	G	H	I
Algoritmo	Tipo de Dado	Métricas	n=100	n=1.000	n=5.000	n=10.000	n=50.000	n=100.000
Busca Linear	Aleatório	ComparaçõesTrocas/Mov.	73,70,0	745,70,0	3.724,30,0	7.456,70,0	36.993,00,0	74.715,00,0
	Ordenado	ComparaçõesTrocas/Mov.	75,00,0	750,30,0	3.771,30,0	7.533,00,0	37.366,30,0	75.583,00,0
Busca Binária	Aleatório	ComparaçõesTrocas/Mov.	5,00,0	8,00,0	11,00,0	12,00,0	14,00,0	15,00,0
	Ordenado	ComparaçõesTrocas/Mov.	5,00,0	8,30,0	11,00,0	12,00,0	14,00,0	15,00,0
Selection Sort	Aleatório	ComparaçõesTrocas/Mov.	4.950,099,0	499.500,0994,0	12.497.500,04.990,0	49.995.000,09.987,0	1.249.975.000,049.988,0	4.999.950.000,099.989,0
	Ordenado	ComparaçõesTrocas/Mov.	4.950,00,0	499.500,00,0	12.497.500,00,0	49.995.000,00,0	1.249.975.000,00,0	4.999.950.000,00,0
Merge Sort	Aleatório	ComparaçõesTrocas/Mov.	548,01.344,0	8.734,019.952,0	55.125,0123.616,0	120.473,0267.232,0	718.266,01.568.928,0	1.536.415,03.337.856,0
	Ordenado	ComparaçõesTrocas/Mov.	356,01.344,0	5.044,019.952,0	32.004,0123.616,0	69.008,0267.232,0	401.952,01.568.928,0	853.904,03.337.856,0

5- Análise e Discussão

a. Teoria vs. prática — o que os dados mostraram

Comparação (dados aleatórios)	Esperado pela teoria	Observado
Selection Sort, n de 10.000 para 50.000 (×5)	≈ ×25 (n^2)	≈ ×29 no tempo
Merge Sort, n de 10.000 para 50.000 (×5)	≈ ×5,9 (n log n)	≈ ×5,3 no tempo
Busca Linear, n de 50.000 para 100.000 (×2)	≈ ×2 (n)	≈ ×1,94 no tempo
Busca Binária, n de 100 a 100.000	cresce como $\log_2 n$	5, 8, 11, 12, 14, 15 comparações

b. Razão de tempo Selection Sort ÷ Merge Sort

n	100	1.000	5.000	10.000	50.000
Aleatório	2,4×	10,2×	17,2×	26,3×	146×
Ordenado	18,2×	12,8×	28,9×	87,3×	392×

A razão cresce com n, como esperado (proporcional a n / log n). Em n = 100.000 a Busca Linear é ≈ 380× mais lenta que a Binária (74.715 contra 15 comparações por consulta).

i. Comparações × tempo nas buscas: a Linear faz ≈ 5.000× mais comparações que a Binária em n = 100.000, mas é só ≈ 380× mais lenta. Que fatores de hardware explicam isso (cache, acesso sequencial vs. disperso)?

ii. Selection Sort tem as mesmas comparações (e tempo parecido: 688 ms contra 698 ms em n = 50.000) em dados ordenados e aleatórios. O que isso diz sobre $\Theta(n^2)$?

iii. Merge Sort em dados ordenados faz ≈ metade das comparações (5.044 contra 8.734 em n = 1.000), mas o mesmo número de movimentos.

iv. Ruído em n pequeno: alguns tempos com n = 100 destoam (ex.: Merge Sort aleatório mais lento que o ordenado). Efeito de JIT/warm-up da JVM; discuta como isso limita as conclusões.

v. Tempos da Busca Binária (≈ 40–140 ns) ficam perto da resolução do *nanoTime()*; nesse caso a contagem de comparações é a evidência mais confiável.

vi. Limitação: a média de comparações por consulta nas buscas usa divisão inteira (trunca).

Resultados para $n = 100.000$ e Desempenho em Escala:

Selection Sort: Levou cerca de **2,77 segundos** tanto para dados aleatórios quanto ordenados, com 5×10^9 comparações. O tempo quadruplicou em relação a $n = 50.000$, confirmando a complexidade $\Theta(n^2)$ e inviabilizando o teste para $n = 500.000$.

Merge Sort: Levou apenas **10,9 ms** (aleatório) e **4,0 ms** (ordenado).

Comparativo: Em $n = 100.000$, o Merge Sort foi **253 vezes mais rápido** em dados aleatórios e **686 vezes mais rápido** em dados ordenados, comprovando a eficiência do $O(n \log n)$ em grande escala.

6- Conclusão e Recomendação

Os testes empíricos confirmaram a teoria assintótica: o Selection Sort ($\Theta(n^2)$) mostrou-se inviável para grandes volumes (2,77 s para $n = 100.000$), enquanto o Merge Sort ($O(n \log n)$) executou a mesma tarefa em apenas 10,9 ms — sendo mais de 250 vezes mais rápido. Nas buscas, a Busca Binária ($\Theta(\log n)$) exigiu no máximo 15 comparações em $n = 100.000$, contra mais de 74.000 da Busca Linear ($\Theta(n)$).

Recomendações para a DataFlow Corp.:

1. Validação de Duplicatas: Adotar a Busca Binária sobre dados mantidos ordenados. Para o futuro, recomenda-se avaliar uma Tabela Hash, que oferece checagem em tempo médio $O(1)$.

2. Ordenação por Distância/Valor: Adotar o Merge Sort (ou Quick Sort/Timsort), eliminando o Selection Sort do ambiente de produção para garantir escalabilidade.

Limitações: O estudo focou-se em dados em memória principal e sofreu pequenas variações de warm-up da JVM em n reduzido. Sugere-se testar cenários com $n > 1.000.000$ e monitorar o consumo de memória em trabalhos futuros.